Curves and Surfaces

Advanced Scripting & Parametrics

Curves and Surfaces

Design Programming 2

Agenda

- A word on final projects
- Back to Transformation Matrices
- Python in grasshopper
- Curves
  - General theory of curves
  - Bezier and NURBS curves
  - Rhino curve functionality
- In 2 weeks Surfaces
  - General theory of surfaces
  - Bezier and NURBS surfaces
  - Rhino curve functionality
  - Surface curvature

Final Projects

Have an initial proposal ready when we get back Feb 24

Curves – tying it all together

Our transform function

$$\begin{bmatrix} x_{new} \\ y_{new} \\ z_{new} \end{bmatrix} = \begin{bmatrix} x1x0 & x1y0 & x1z0 \\ y1x0 & y1y0 & y1z0 \\ z1x0 & z1y0 & z1z0 \end{bmatrix} \mathbf{x} \begin{bmatrix} x_{old} \\ y_{old} \\ z_{old} \end{bmatrix}$$

Generalized

$$\begin{bmatrix} x_{new} \\ y_{new} \\ z_{new} \end{bmatrix} = \begin{bmatrix} \text{Some complicated function}_x(x_{old}, y_{old}, z_{old}) \\ \text{Some complicated function}_y(x_{old}, y_{old}, z_{old}) \\ \text{Some complicated function}_z(x_{old}, y_{old}, z_{old}) \end{bmatrix} \mathbf{x} \begin{bmatrix} x_{old} \\ y_{old} \\ z_{old} \end{bmatrix}$$

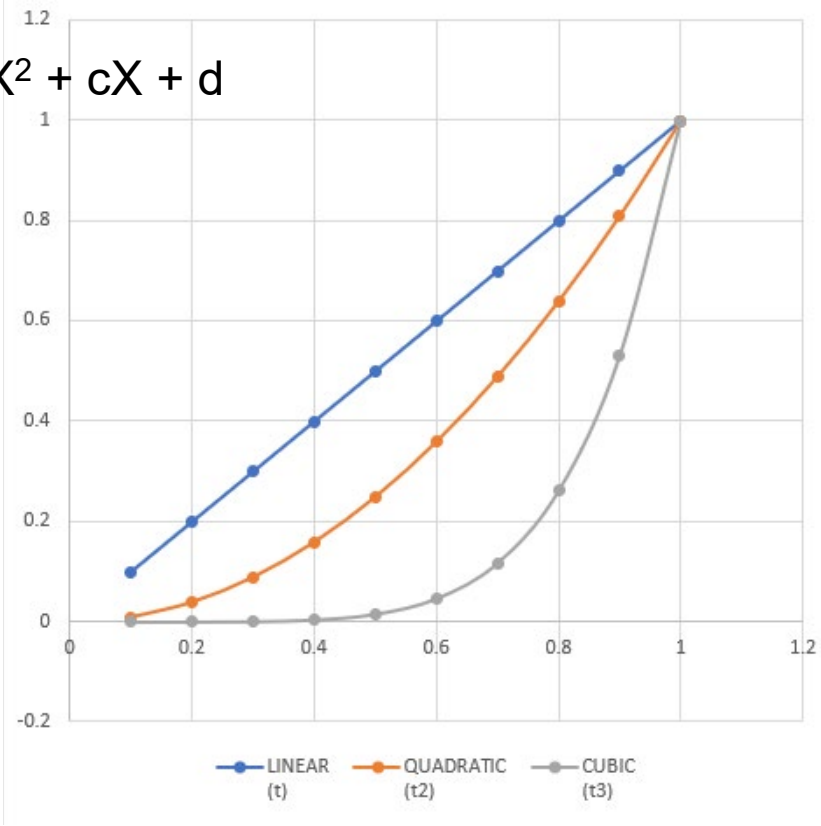
Now: Curves

$$\begin{bmatrix} x_{new} \\ y_{new} \\ z_{new} \end{bmatrix} = \begin{bmatrix} Fx(t) \\ Fy(t) \\ Fz(t) \end{bmatrix} \mathbf{x} \begin{bmatrix} t \end{bmatrix}$$

T	LINEAR (t)	QUADRATIC (t^2)	CUBIC (t^3)
0.1	0.1	0.01	0.000001
0.2	0.2	0.04	0.000064
0.3	0.3	0.09	0.000729
0.4	0.4	0.16	0.004096
0.5	0.5	0.25	0.015625
0.6	0.6	0.36	0.046656
0.7	0.7	0.49	0.117649
0.8	0.8	0.64	0.262144
0.9	0.9	0.81	0.531441
1	1	1	1

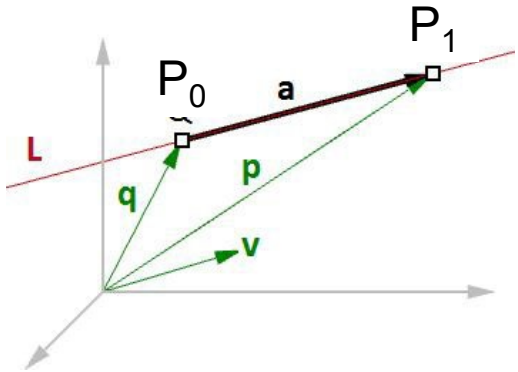
$Y = aX + b$

$Y = aX^{3 + b}X^2 + cX + d$



Vector equation of ~~line –~~ 2 point curve

The vector line equation is used in 3D modeling applications and computer graphics.



If we know the direction of a line and a point on that line, then we can find any other point on the line using vectors, as in the following:

L = line
v = <a, b, c> line direction unit vector
Q = (x0, y0, z0) line position point
P = (x, y, z) any point on the line
P = Q + *t* * **v**

Given a point Q and a direction v on a line, any point P on that line can be calculated using the vector equation of a line C = Q + t * v where *t* is a number.

$$C_t = P_0 + t * (P_1 - P_0)$$

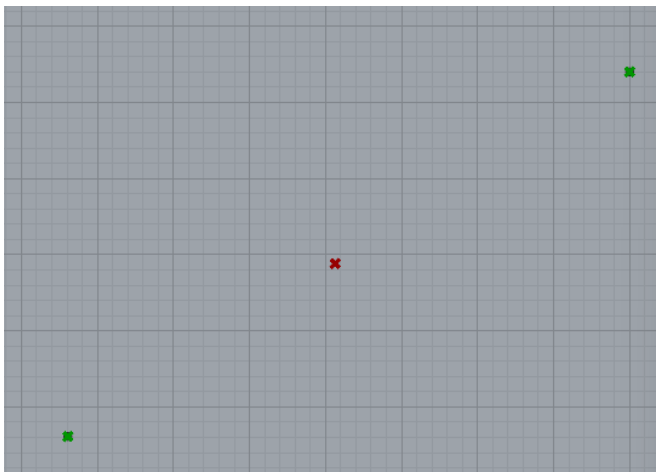
t=0: P_0

t=1: P_1

t=0.5: $\frac{(P_1 + P_0)}{2}$ #midpoint

Curves as a blending function

Given points p1, p2, and t = [0…1]



$C_t = 1*P_0 + t * (P_1 - P_0)$

$C_t = (1-t) * P_0 \quad + \quad t * P_1$

OR

$C_t = (1-t)*P0 + t*P1$

$C_{t=0} = (1)*P0 + 0*P1$

$C_{t=1} = \cancel{(1-t)*P0 +} \cancel{t*}P1$

$C_{t=0.5} = (1-0.5)*P0 + 0.5*P1$

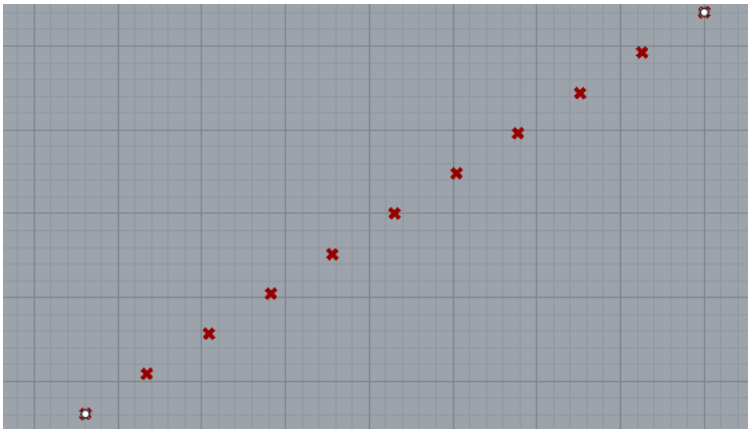
$$= \begin{pmatrix} (1-t)*p0_x + t*p1_x \\ (1-t)*p0_y + t*p1_y \\ (1-t)*p0_z + t*p1_z \end{pmatrix}$$

t=0: P_0

t=1: P_1

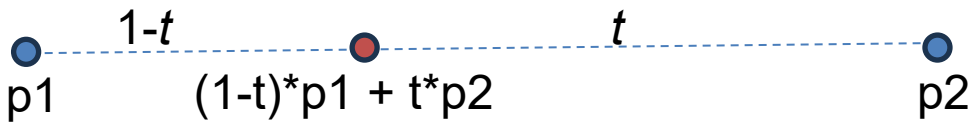
t=0.5: $\frac{(P_1 + P_0)}{2}$ #midpoint

Try it yourself – a list of points

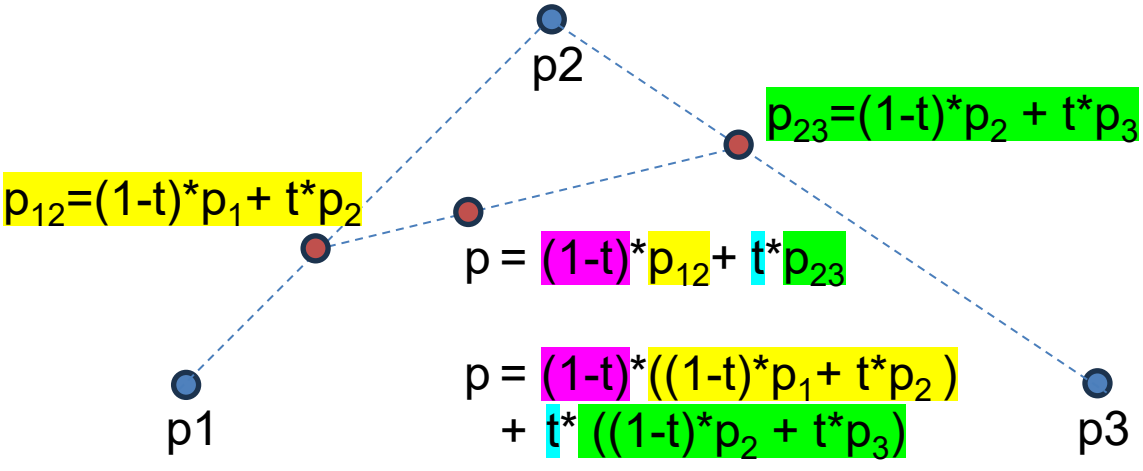


```
import rhinoscriptsyntax as rs
import scriptcontext as sc
import math
import numpy

import System
import System.Collections.Generic
import Rhino

c = []
ts = numpy.arange(0,1.1, 0.1)

#add the rest here
```

Curves as a blending function: 2 points



Curves as a blending function: 3 points!



$$p = (1-t)^2 * p_1 + 2t(1-t) * p_2 + t^2 * p_3$$

Bernstein Polynomials

The *n*+1 **Bernstein basis polynomials** of degree *n* are defined as

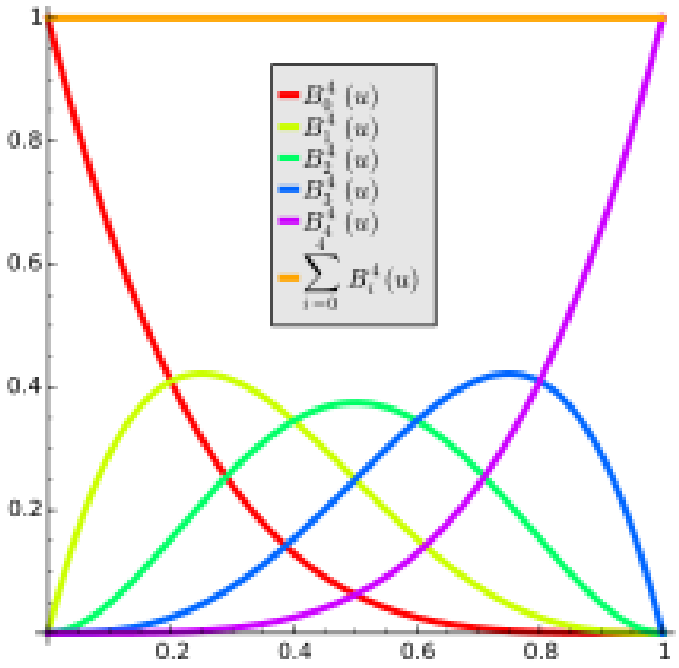
$$b_{\nu,n}(x) := \binom{n}{\nu} x^{\nu}(1-x)^{n-\nu}, \quad \nu = 0, \ldots, n,$$

where $\binom{n}{\nu}$ is a binomial coefficient.

So, for example, $b_{2,5}(x) = \binom{5}{2} x^2(1-x)^3 = 10x^2(1-x)^3$.

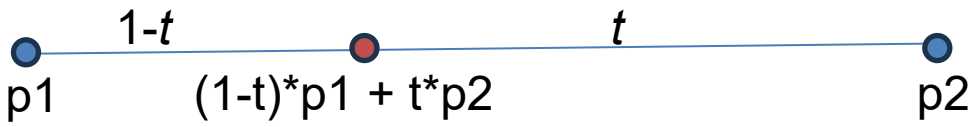
The first few Bernstein basis polynomials for blending 1, 2, 3 or 4 values together are:

$$\begin{aligned}
&b_{0,0}(x) = 1, \\
&b_{0,1}(x) = 1 - x, && b_{1,1}(x) = x \\
&b_{0,2}(x) = (1-x)^2, && b_{1,2}(x) = 2x(1-x), && b_{2,2}(x) = x^2 \\
&b_{0,3}(x) = (1-x)^3, && b_{1,3}(x) = 3x(1-x)^2, && b_{2,3}(x) = 3x^2(1-x), && b_{3,3}(x) = x^3
\end{aligned}$$



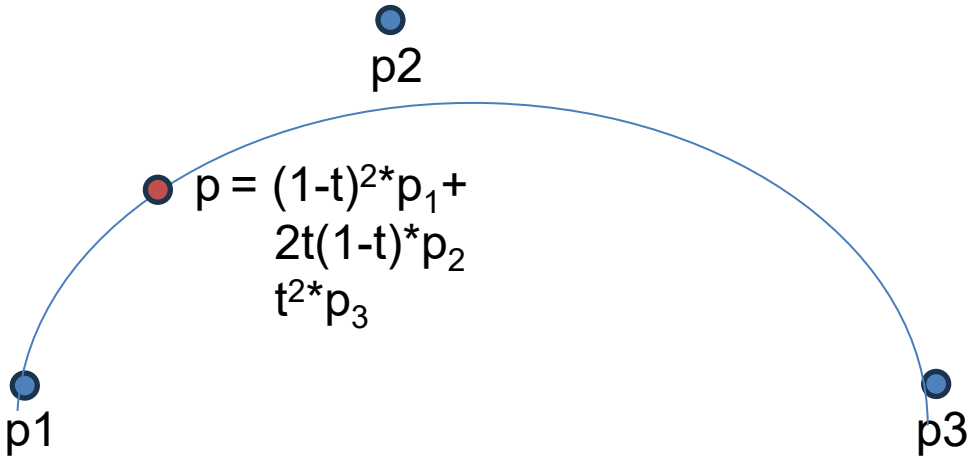
A LINEAR BEZIER Curve
Order = 1 = #points(2) - 1

$$b_{0,1}(x) = 1 - x, \qquad b_{1,1}(x) = x$$



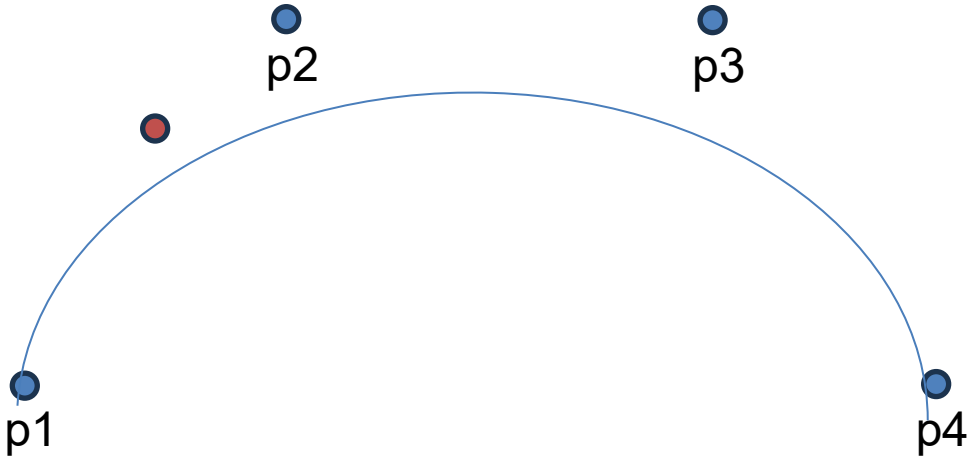
A QUADRATIC BEZIER Curve
Order = 2 = #points(3) - 1

$$b_{0,2}(x) = (1 - x)^2, \qquad b_{1,2}(x) = 2x(1 - x), \qquad b_{2,2}(x) = x^2$$



A CUBIC BEZIER Curve
Order = 3 = #points(4) - 1

$$b_{0,3}(x) = (1 - x)^3, \qquad b_{1,3}(x) = 3x(1 - x)^2,$$
$$b_{2,3}(x) = 3x^2(1 - x), \qquad b_{3,3}(x) = x^3$$



NURBS Curves

NURBS is an accurate mathematical representation of curves that is highly intuitive to edit. It is easy to represent free-form curves using NURBS and the control structure makes it easy and predictable to edit.

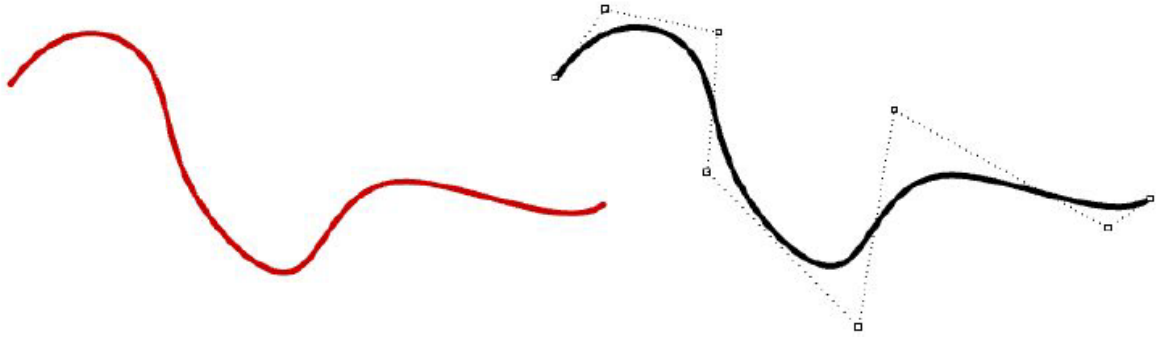


Figure (36): Non-uniform rational B-splines and their control structure.

There are many books and references for those of you interested in an in-depth reading about NURBS[6]. A basic understanding of NURBS is however necessary to help use a NURBS modeler more effectively. There are four main attributes define the NURBS curve: *degree*, *control points*, *knots*, and *evaluation rules*.

Weights of control points

Each control point has an associated number called *weight*. With a few exceptions, weights are positive numbers. When all control points have the same weight, usually 1, the curve is called non-rational. Intuitively, you can think of weights as the amount of gravity each control point has. The higher the relative weight a control point has, the closer the curve is pulled towards that control point.

Knots

Each NURBS curve has a list of numbers associated with it called a *knots* (sometimes referred to as *knot vector*). Knots are a little harder to understand and set. While using a 3-D modeler, you will not need to manually set the knots for each curve you create; a few things will be useful to learn about knots.

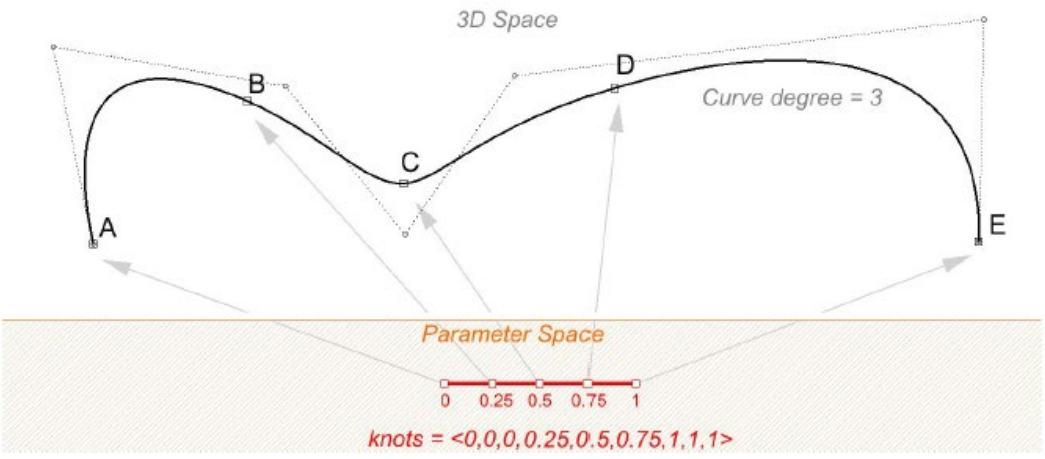
Knots are parameter values

Knots are a non-decreasing list of parameter values that lie within the curve domain. In Rhino, there is degree-1 more knots than control points. That is the number of knots equals the number of control points plus curve degree minus 1:

|knots| = |CVs| + Degree - 1

Usually, for non-periodic curves, the first degree many knots are equal to the domain minimum, and the last degree many knots are equal to the domain maximum.

For example, the knots of an open degree-3 NURBS curve with 7 control points and a domain between 0 and 4 may look like <0, 0, 0, 1, 2, 3, 4, 4, 4>.



Properties of Bezier / NURBS Curves

The sum of the "blending parameters" at each value of *t* = 1
These parameters vary for each point from t = [0…1]

c at the p_0 is tangent to the first segment of the "control polygon"
c at the p_{LAST} is tangent to the last segment of the control polygon

Every point on *c* lies within the control polygon

Multiplicity of points reduces the order of the curve at that point
From 3 (cubic)
To 2 (quadratic)
To 1 (linear)
To 0 (point)

In Rhino, the domain of a curve is not [0…1],
it will be [0…something else]

A *periodic curve* is a curve that is continuous from end of curve back to the beginning of the curve (it's "closed')

Curve API

- Curve
  - Constructors
  - Properties
    - Degree
    - Dimension
    - Disposed
    - Domain
    - HasBrepForm
    - HasUserData
    - IsClosed
    - IsDeformable
    - IsDocumentControlled
    - IsDocumentControlled
    - IsPeriodic
    - IsSubDFriendly
    - IsValid
    - ObjectType
    - PerformCorruptionTesting
    - PointAtEnd
    - PointAtMid
    - PointAtStart
    - SpanCount
    - TangentAtEnd
    - TangentAtStart
    - UserData
    - UserDictionary
    - UserStringCount
  - Methods
    - ChangeClosedCurveSeam
    - ChangeDimension
    - ClosedCurveOrientation
    - ClosestPoint
    - ClosestPoints
    - CombineShortSegments
    - ComponentIndex
    - ConstructConstObject
    - Contains
    - ControlPolygon
    - CreateArcBlend
    - CreateArcCornerRectangle
    - CreateArcLineArcBlend
    - CreateBlendCurve
    - CreateBooleanDifference
    - CreateBooleanIntersection
    - CreateBooleanRegions
    - CreateBooleanUnion
    - CreateConicCornerRectangle
    - CreateControlPointCurve
    - CreateCurve2View
    - CreateFillet
    - CreateFilletCornersCurve
    - CreateFilletCurves
    - CreateInterpolatedCurve
    - CreateMatchCurve
    - CreateMeanCurve
    - CreatePeriodicCurve
    - CreateRevisionCloud
    - CreateSoftEditCurve
    - CreateTextOutlines
    - CreateTweenCurves
    - CreateTweenCurvesWithMatching
    - CreateTweenCurvesWithSampling
    - CurvatureAt
    - DataCRC
    - DeleteAllUserStrings
    - DeleteUserString
    - DerivativeAt
    - Dispose
    - Dispose
    - DivideByCount
    - DivideByLength
    - DivideEquidistant
    - DoDirectionsMatch
    - Duplicate
    - Duplicate
    - DuplicateCurve
    - DuplicateSegments
    - DuplicateShallow
    - EnsurePrivateCopy
    - Extend
    - ExtendByArc
    - ExtendByLine
    - ExtendOnSurface
    - ExtremeParameters
    - Fair
    - FilletSurfaceToCurve
    - FilletSurfaceToRail
    - FindLocalInflection
    - Fit
    - FrameAt
    - FromBase64String
    - FromJSON
    - GeometryEquals
    - GeometryReferenceEquals
    - GetBoundingBox
    - GetConicSectionType
    - GetCurveParameterFromNurbsFormParameter
    - GetDistancesBetweenCurves
    - GetFilletPoints
    - GetLength
    - GetLocalPerpPoint
    - GetLocalTangentPoint
    - GetNextDiscontinuity
    - GetNurbsFormParameterFromCurveParameter
    - GetObjectData
    - GetPerpendicularFrames
    - GetSubCurves
    - GetUserString
    - GetUserStrings
    - HasNurbsForm
    - InflectionPoints

NurbsCurve API

- NurbsCurve
  - Constructors
  - Properties
    - Degree
    - Dimension
    - Disposed
    - Domain
    - HasBezierSpans
    - HasBrepForm
    - HasUserData
    - IsClosed
    - IsDeformable
    - IsDocumentControlled
    - IsDocumentControlled
    - IsPeriodic
    - IsRational
    - IsSubDFriendly
    - IsValid
    - Knots
    - ObjectType
    - Order
    - PerformCorruptionTesting
    - PointAtEnd
    - PointAtMid
    - PointAtStart
    - Points
    - SpanCount
    - TangentAtEnd
    - TangentAtStart
    - UserData
    - UserDictionary
    - UserStringCount
  - Methods
    - Append
    - ChangeClosedCurveSeam
    - ChangeDimension
    - ClosedCurveOrientation
    - ClosestPoint
    - ClosestPoints
    - CombineShortSegments
    - ComponentIndex
    - ConstructConstObject
    - Contains
    - ControlPolygon
    - ConvertSpanToBezier
    - Create
    - CreateArcBlend
    - CreateArcCornerRectangle
    - CreateArcLineArcBlend
    - CreateBlendCurve
    - CreateBooleanDifference
    - CreateBooleanIntersection
    - CreateBooleanRegions
    - CreateBooleanUnion
    - CreateConicCornerRectangle
    - CreateControlPointCurve
    - CreateCurve2View
    - CreateFillet
    - CreateFilletCornersCurve
    - CreateFilletCurves
    - CreateFromArc
    - CreateFromCircle
    - CreateFromEllipse
    - CreateFromFitPoints
    - CreateFromLine
    - CreateHSpline
    - CreateInterpolatedCurve
    - CreateMatchCurve
    - CreateMeanCurve
    - CreateNonRationalArcBezier
    - CreateParabolaFromFocus
    - CreateParabolaFromPoints
    - CreateParabolaFromVertex
    - CreatePeriodicCurve
    - CreatePlanarRailFrames
    - CreateRailFrames
    - CreateRevisionCloud
    - CreateSoftEditCurve
    - CreateSpiral
    - CreateSubDFriendly
    - CreateTextOutlines
    - CreateTweenCurves
    - CreateTweenCurvesWithMatchir
    - CreateTweenCurvesWithSamplir
    - CurvatureAt
    - DataCRC
    - DeleteAllUserStrings
    - DeleteUserString
    - DerivativeAt
    - Dispose
    - Dispose
    - Dispose
    - DivideAsContour
    - DivideByCount
    - DivideByLength
    - DivideEquidistant
    - DoDirectionsMatch
    - Duplicate
    - Duplicate
    - DuplicateCurve
    - DuplicateSegments
    - DuplicateShallow
    - EnsurePrivateCopy
    - EpsilonEquals
    - Extend
    - ExtendByArc
    - ExtendByLine
    - ExtendOnSurface
    - ExtremeParameters
    - Fair
    - FilletSurfaceToCurve
    - FilletSurfaceToRail
    - FindLocalInflection
    - Fit
    - FrameAt
    - FromBase64String

LineCurve API

- LineCurve
  - Constructors
  - Properties
    - Degree
    - Dimension
    - Disposed
    - Domain
    - HasBrepForm
    - HasUserData
    - IsClosed
    - IsDeformable
    - IsDocumentControlled
    - IsDocumentControlled
    - IsPeriodic
    - IsSubDFriendly
    - IsValid
    - Line
    - ObjectType
    - PerformCorruptionTesting
    - PointAtEnd
    - PointAtMid
    - PointAtStart
    - SpanCount
    - TangentAtEnd
    - TangentAtStart
    - UserData
    - UserDictionary
    - UserStringCount
  - Methods
    - ChangeClosedCurveSeam
    - ChangeDimension
    - ClosedCurveOrientation
    - ClosestPoint
    - ClosestPoints
    - CombineShortSegments
    - ComponentIndex
    - ConstructConstObject
    - Contains
    - ControlPolygon
    - CreateArcBlend
    - CreateArcCornerRectangle
    - CreateArcLineArcBlend
    - CreateBlendCurve
    - CreateBooleanDifference
    - CreateBooleanIntersection
    - CreateBooleanRegions
    - CreateBooleanUnion
    - CreateConicCornerRectangle
    - CreateControlPointCurve
    - CreateCurve2View
    - CreateFillet
    - CreateFilletCornersCurve
    - CreateFilletCurves
    - CreateInterpolatedCurve
    - CreateMatchCurve
    - CreateMeanCurve
    - CreatePeriodicCurve
    - CreateRevisionCloud
    - CreateSoftEditCurve
    - CreateTextOutlines
    - CreateTweenCurves
    - CreateTweenCurvesWithMatching
    - CreateTweenCurvesWithSampling
    - CurvatureAt
    - DataCRC
    - DeleteAllUserStrings
    - DeleteUserString
    - DerivativeAt
    - Dispose
    - Dispose
    - DivideAsContour
    - DivideByCount
    - DivideByLength
    - DivideEquidistant
    - DoDirectionsMatch
    - Duplicate
    - Duplicate
    - DuplicateCurve
    - DuplicateSegments
    - DuplicateShallow
    - EnsurePrivateCopy
    - Extend
    - ExtendByArc
    - ExtendByLine
    - ExtendOnSurface
    - ExtremeParameters
    - Fair
    - FilletSurfaceToCurve
    - FilletSurfaceToRail
    - FindLocalInflection
    - Fit
    - FrameAt
    - FromBase64String
    - FromJSON
    - GeometryEquals
    - GeometryReferenceEquals
    - GetBoundingBox
    - GetConicSectionType
    - GetCurveParameterFromNurbsFormParameter
    - GetDistancesBetweenCurves
    - GetFilletPoints
    - GetLength
    - GetLocalPerpPoint
    - GetLocalTangentPoint
    - GetNextDiscontinuity
    - GetNurbsFormParameterFromCurveParameter
    - GetObjectData
    - GetPerpendicularFrames
    - GetSubCurves
    - GetUserString
    - GetUserStrings

Classes - redoux

Recall:
Class: the category of instances
Instance: a specific example of the category

Properties

Properties tend to be of the instance, and are data contained by the instance. They are accessed as variables:

myThing.name

crv.Degree

Methods

Methods are functions of the class or the instance. There are two types:
Instance methods – send and receive messages to the instance.

crv.ControlPolygon() #gets the curve polygon

Class ("Static") methods: send and receive messages from the class

Description:

Gets the curve's control polygon.

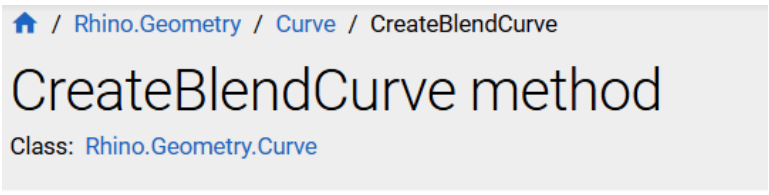
Syntax:

```
public Polyline ControlPolygon()
```

Returns:

Type: Polyline
The control polygon as a polyline if successful, None otherwise.

Available since:
8.11



Description:

Makes a curve blend between 2 curves at the parameters specified with t

Syntax:

```
public static Curve CreateBlendCurve(
  Curve curve0,
  double t0,
  bool reverse0,
  BlendContinuity continuity0,
  Curve curve1,
  double t1,
  bool reverse1,
  BlendContinuity continuity1
)
```

Exercise: create a script that accesses curve instance and class functions

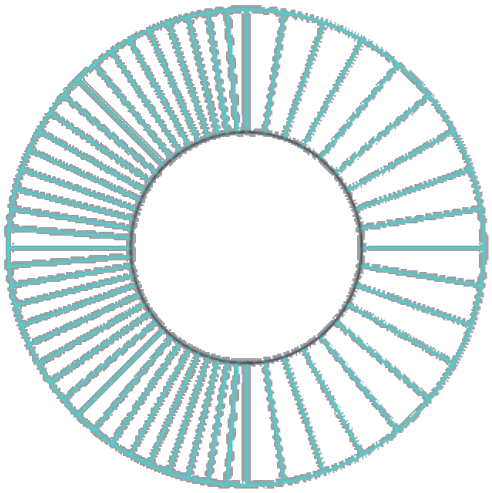
Curvature and Curve Frames

For a circle, this is easy:

curvature = 1/radius

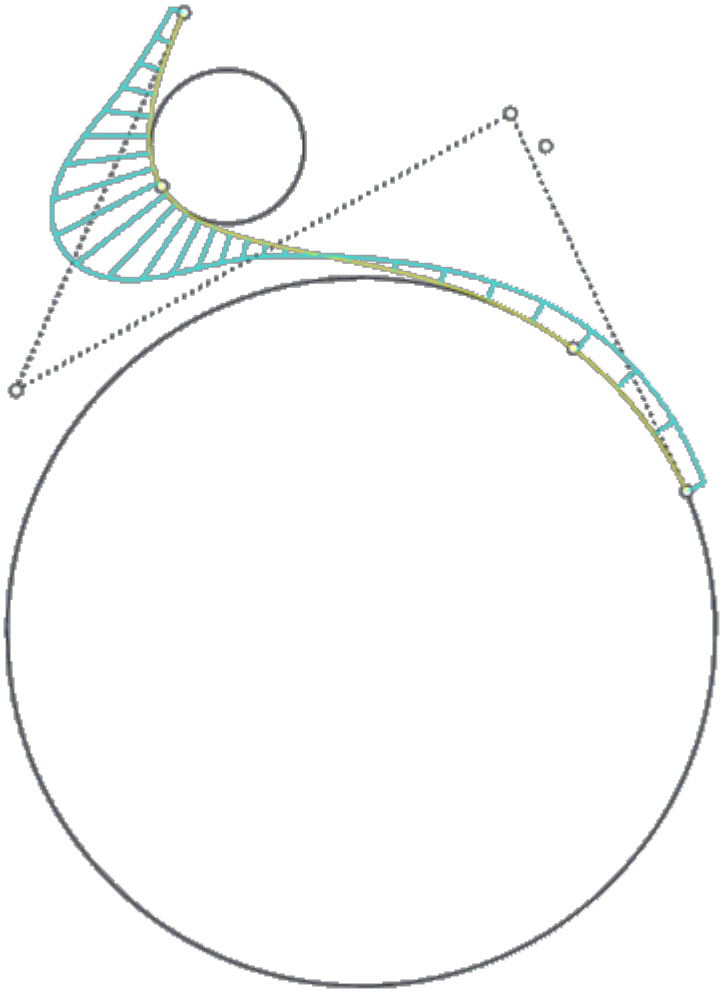
For a line, this is also easy
curvature = 0

What is the radius?



For more general curves, curvature varies continuously at each point, and is 1/the “effective radius” at that point.

See **CurvatureGraph**
and **Curvature**



Curve Frames

The local **Frame** of a curve at a point on the curve is defined by

- The **tangent** vector at the point
- The **normal** to the curve – in the direction of the curvature circle
- The **binormal** of the curve – the vector perpendicular to both the tangent and normal